Universidade Óscar Ribas
Faculdade De Ciências e Tecnologia
Engenharia Informática e Comunicações

Analise de Algortimo e Estrutura de Dados

Estrutura de dados não lineares: Arvore Binaria , Grafos, Dicionario

Integrante do Grupo

Vanira Xamiquelengue

Docente

_____

Divava

Luanda, 26 Maio de 2026

INTRODUÇÃO

Estruturas de dados lineares como arrays, listas ligadas, pilhas e filas, onde os elementos estão organizados de forma sequencial e cada item possui um único predecessor e sucessor direto, as estruturas de dados não linear**es** organizam a informação de forma hierárquica ou interconectada. Essa abordagem é fundamental na ciência da computação para representar relações complexas entre dados, otimizar processos de busca e espelhar situações do mundo real que a linearidade simplesmente não consegue comportar de maneira eficiente.

CONTEXTUALIZACÃO

1. Árvores Binárias

Conceito

É uma estrutura de dados não linear e hierárquica na qual cada elemento (chamado de nó) pode ter, no máximo, dois nós filhos. Por convenção, esses filhos são denominados filho da esquerda e filho da direita. O nó topo da hierarquia é a raiz. Se um nó não possui filhos, é chamado de nó folha. Esta limitação de no máximo dois caminhos por nó simplifica a lógica de percurso e otimiza as operações de busca.

Representação

A nível de código e memória, uma árvore binária é tipicamente representada através de alocação dinâmica (encadeamento), onde cada nó é um objeto ou estrutura composta por três partes.

Exemplo Prático

```
public class ArvoreVanira {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here

        Arvore arvore = new Arvore();

        arvore.raiz = arvore.inserirFilho(arvore.raiz, 30);
        arvore.inserirFilho(arvore.raiz, 0);
        arvore.inserirFilho(arvore.raiz, 3);
        arvore.inserirFilho(arvore.raiz, 12);
        arvore.inserirFilho(arvore.raiz, -9);
        arvore.inserirFilho(arvore.raiz, 5);

        arvore.percorrerArvore(arvore.raiz);
    }

}
```

```java
 * @author qiu
 */
public class NoArvore {
    int valor;
    NoArvore filhoEsquerda;
    NoArvore filhoDireita;

    public NoArvore(int valor){
        this.valor = valor;
    }
}
```

2. Grafos

Conceito

Um Grafo é uma estrutura de dados não linear mais geral, composta por um conjunto de pontos chamados Vértices (ou nós) e por linhas que unem esses pontos, chamadas Arestas (ou arcos). Ao contrário das árvores, os grafos não têm uma raiz e podem conter ciclos (caminhos que voltam ao ponto de partida). Eles podem ser direcionados (quando as arestas têm um sentido definido, como uma rua de sentido único) ou não direcionados.

Aplicação

Os grafos são utilizados para modelar redes complexas no mundo real. As aplicações mais comuns incluem:

Redes Sociais: Onde os utilizadores são os vértices e as amizades/conexões são as arestas.

Sistemas de GPS e Mapas: Onde as localizações ou cidades são os vértices e as estradas são as arestas (frequentemente com "pesos", que representam a distância ou o trânsito).

Redes de Computadores: Para mapear o fluxo de dados entre servidores e roteadores.

Exemplo Prático

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

// 1. Classe que representa a ligação (Aresta) entre as cidades
class Aresta {
    String destino;
    int peso; // Distância em Quilómetros (km)

    public Aresta(String destino, int peso) {
        String.valueOf(this.destino = destino);
        this.peso = peso;
    }
}
```

```java
// 2. Classe que representa a estrutura do Grafo
class Grafo {
    // Lista de adjacência: cada cidade (Chave) tem uma lista de ligações (Valor)
    private Map<String, List<Aresta>> listaAdjacencia;

    public Grafo() {
        this.listaAdjacencia = new HashMap<>();
    }

    // Método para adicionar uma cidade (Vértice) ao grafo
    public void adicionarCidade(String cidade) {
        listaAdjacencia.putIfAbsent(cidade, new ArrayList<>());
    }

    // Método para conectar duas cidades (Aresta Bidirecional / Não Direcionado)
    public void adicionarLigacao(String origem, String destino, int distancia) {
        // Garante que as cidades existem no grafo
        adicionarCidade(origem);
        adicionarCidade(destino);

        // Como o camião pode ir e voltar, a ligação existe nos dois sentidos
        listaAdjacencia.get(origem).add(new Aresta(destino, distancia));
        listaAdjacencia.get(destino).add(new Aresta(origem, distancia));
    }

    // Método para exibir a estrutura do grafo na consola
```

3. Dicionários (Mapas ou Tabelas Hash)

Conceito

Um Dicionário (também conhecido como Map ou Tabela Hash) é uma estrutura de dados que armazena uma coleção de elementos baseada no conceito de Chave-Valor. Cada elemento possui uma chave única que serve como identificador e um valor associado a essa chave. Embora a

estrutura lógica de busca pareça direta, a organização interna dos dados (geralmente gerida por uma função de espalhamento ou hash) não segue uma ordem linear sequencial, classificando-a como uma estrutura não linear.

Aplicação

Os dicionários são aplicados sempre que existe a necessidade de realizar buscas, inserções e remoções de forma extremamente rápida.

Exemplo Prático

```
lic class DicionarioVanira {

 /**
  * @param args the command line arguments
  */
 public static void main(String[] args) {
     // TODO code application logic here

     HashMap<String, String> estudante = new HashMap<>();

     estudante.put("nome", "Regina");
     estudante.put("sexo", "F");
     estudante.put("nascimento", "20-10-2***");

     System.out.println(estudante.get("nome"));
     System.out.println(estudante.get("sexo"));
     System.out.println(estudante.get("nascimento"));
 }
```

CONCLUSÃO

O estudo das estruturas de dados não linearescom particular enfoque nas Árvores Binárias, Grafos e Dicionários evidenciam que a linearidade sequencial é insuficiente para responder à complexidade e ao volume de dados exigidos pelos sistemas de software contemporâneos. Enquanto as estruturas lineares servem propósitos de armazenamento simples, as não lineares surgem como uma resposta estratégica para modelar relações complexas, hierarquias e redes interconectadas que espelham o mundo real.

Em suma, conclui-se que a escolha e a implementação correta de uma estrutura de dados não linear não representam apenas uma decisão de programação, mas sim um fator determinante para a escalabilidade, estabilidade e performance de qualquer solução tecnológica. Compreender os seus conceitos, representações e aplicações práticas é, por isso, um requisito obrigatório para o desenvolvimento de sistemas robustos, eficientes e preparados para os desafios da engenharia de software atual.